

Name : P.S.Thilakraj

Regno: 192412434

Polyphase Decimator Design

Simulate a polyphase decimator for a 32-tap FIR filter and compare its efficiency with direct

implementation. (25) (BL4) (WK4)

Given Data:

- FIR Length = 32 taps
- Decimation Factor = 4

Tasks:

1. Design the FIR filter.
2. Decompose it into polyphase components.
3. Implement the polyphase decimator.
4. Compare output with direct implementation.
5. Count arithmetic operations.
6. Evaluate computational savings.

Code :

```
%% Polyphase Decimator Design
```

```
% Parameters
```

```
M = 4;          % Decimation factor
```

```
N = 32;         % FIR length
```

```
Fs = 1e6;       % Input sampling rate (Hz)
```

```
Fc = 0.2;       % Normalized cutoff (relative to Nyquist)
```

```
%% 1. Design the FIR filter
```

```
h = fir1(N-1, Fc); % 32-tap lowpass FIR filter
```

```
disp('FIR Coefficients:'); disp(h.);
```

```
%% 2. Decompose into polyphase components
```

```
% Reshape coefficients into M polyphase branches
```

```
E = reshape(h, M, []); % Each row = one polyphase branch
```

```
disp('Polyphase Components:'); disp(E);
```

```
%% 3. Implement polyphase decimator
```

```
% Generate input signal (bandlimited + noise)
```

```
t = 0:1/Fs:0.01;
```

```
x = cos(2*pi*0.05*Fs*t) + 0.5*randn(size(t));
```

```
% Direct implementation
```

```
y_direct = downsample(filter(h,1,x), M);
```

```
% Polyphase implementation
```

```
% Use MATLAB's built-in dsp.FIRDecimator for efficiency
```

```
polyDec = dsp.FIRDecimator(M, h);
```

```
y_poly = polyDec(x.');
```

```
%% 4. Compare outputs
```

```
figure;
```

```
subplot(2,1,1); plot(y_direct); title('Direct Decimation Output');
```

```
subplot(2,1,2); plot(y_poly); title('Polyphase Decimation Output');
```

```
%% 5. Count arithmetic operations
```

```
% Direct implementation: N multiplications per output sample
```

```
ops_direct = N;
```

```
% Polyphase implementation: N/M multiplications per output sample
```

```
ops_poly = N/M;
```

```
fprintf('Direct implementation: %d multiplications per output sample\n', ops_direct);  
fprintf('Polyphase implementation: %d multiplications per output sample\n', ops_poly);
```

%% 6. Evaluate computational savings

```
saving = (1 - ops_poly/ops_direct)*100;
```

```
fprintf('Computational savings: %.2f %%\n', saving);
```

Output :

